

ENIDS: A Deep Learning-Based Ensemble Framework for Network Intrusion Detection Systems

Ibrahim Mohammed Sayem, Moinul Islam Sayed, Sajal Saha, and Anwar Haque

Abstract—Rapid and widespread adoption of emerging Information Technology (IT) infrastructures and services in commercial and private endeavors opens new horizons for novel cyberattacks. Network Intrusion Detection Systems (NIDS) gained attention as an effective means of combating various cyber threats. Recent research demonstrates the potency of machine learning (ML) and deep learning (DL) approaches in the development of NIDS. In this paper, we propose a DL-based framework called the Ensemble Framework for Network Intrusion Detection System (ENIDS) to detect various types of cyberattacks, which includes dynamic data pre-processing, optimal feature selection, the handling of imbalanced data samples, and a DL-based ensemble model. Our DL-based ensemble model is comprised of two layers: the base learner and the meta-learner. The base learner is composed of three robust DL models: convolutional neural networks (CNN), long short-term memory (LSTM), and gated recurrent units (GRU), and the meta-learner is a deep neural network (DNN) model. The proposed framework experimented with two publicly available and popular network traffic datasets, namely UNSW-15 and CICIDS-2017. In the UNSW-15 and CICIDS-2017 datasets, our proposed framework detects cyberattacks with an accuracy of 90.6% and 99.6% and an F1-score of 90.5% and 99.6%, respectively. According to experimental findings, the proposed ensemble framework outperforms existing state-of-the-art approaches and demonstrates better performance than benchmark DL methods in terms of accuracy, F1-score, and execution time for training and testing.

Index Terms—Cyber Security, Data Resampling, Deep Learning, Ensemble Learning, Intrusion Detection System

I. INTRODUCTION

IT infrastructures have been expanding rapidly in capacity, system loads, and complexity in recent years. People's lives are becoming more reliant on the applications of smart appliances, smartphones, and cloud services. Particularly, when the COVID-19 outbreak began in 2020, the internet became a lifeline for people in practically every industry. According to the Pew Research Center, 90% of Americans regarded various online tools as a vital part of their daily lives during the coronavirus pandemic, with most of them using those tools for business, video meetings, academics, and communication purposes [1]. As per the 2024 global overview report of DataReportal, about 5.35 billion individuals uses the Internet,

which is more than 66% of the world's population [2]. Besides that, Ericsson foresees that the widespread adoption of 5G and its enhanced features will result in a rise in 5G subscribers, surpassing 5.3 billion by 2029, which will comprise 58% of overall mobile users [3]. This rapid expansion of the users, network infrastructures, and protocols of emerging technologies is posing security issues and opening horizons for novel threats [4]. 5G networks, for example, provide greater bandwidth and reduced latency but expand the attack surface, increasing system vulnerabilities [5].

Any offensive action conducted towards diverse systems to access, steal, or modify valuable data, install malicious software, or interrupt routine services without the administrator's consent is considered a cyberattack. Any cyberattack can be categorized as targeted or untargeted. Targeted attacks attempt to infiltrate specific businesses, while untargeted attacks breach as many systems or devices as possible. Targeted cyberattacks include spear-phishing and deploying botnets, while untargeted attacks include ransomware and phishing [6]. Malware, phishing, denial-of-service (DoS), distributed denial-of-service (DDoS), reconnaissance, backdoors, etc. are some common types of cyberattacks.

Cyberattacks on both the core network and host levels have increased in recent years, disrupting a broad range of business and government domains. According to Check Point Research, the weekly incidence of cyberattacks soared to 50% during the COVID-19 pandemic, with education and research being the most afflicted sectors [7]. In 2021, a large-scale cyberattack hampered nine US government agencies, where attackers deployed untraceable backdoors in Microsoft Exchange and compromised access to the server to gain all information residing inside the server [8]. According to CSIS (Center for Strategic and International Studies), a Russian hacker group launched an "information attack" in February 2022 to obtain access to two of Ukraine's financial institutions, causing service outages [9]. Throughout the first half of 2021, 5,591 network layer DDoS attacks were reported, according to the "2021 DDoS Threat Landscape Report," and the attack technique is changing in dimension, volume, regularity, and complexity [10].

In order to detect and prevent cyberattacks on the networks, firewalls are used as the primary defense mechanism, and a network intrusion detection system, or NIDS, is employed as an advanced security mechanism. NIDS is mainly a security application that can analyze network traffic, detect security threats, and take measures when an anomaly is detected.

Ibrahim Mohammed Sayem, Moinul Islam Sayed, and Anwar Haque are with the Department of Computer Science, Western University, London, ON Canada N6A 5B7. (e-mail: isayem@uwo.ca, msayed6@uwo.ca, ahaque32@uwo.ca)

Sajal Saha is with the Department of Computer Science, University of Northern British Columbia, Prince George, BC Canada V2N 4Z9. (e-mail: sajal.saha@unbc.ca)

Based on Donghwoon et al. [11] an intrusion detection system (IDS) should consist of three main steps: First, the IDS needs to collect and track network flow data. After that, the IDS needs to reformat and reshape the network data by cleaning the raw data, processing them, and making them feasible in the next step. Finally, using the processed data as input, a model will determine if network traffic is normal or possesses any abnormalities. Developing an effective and efficient model is the most crucial part of building a NIDS. IDS are categorized as signature-based and anomaly-based. Traditional signature-based IDS compares network packets to known signatures. In contrast, models in anomaly-based IDS produced by integrating ML and DL are capable of learning attack signatures to create the attack pattern. Because of the dynamic nature of diverse cyberattack patterns, anomaly-based IDS is more viable than traditional IDS. However, DL techniques outperform shallow ML-based approaches in the case of high-dimensional and substantially large datasets [12].

Many previous studies utilized UNSW-15 [13] and CICIDS-2017 [14] to detect network abnormalities; however, the majority followed binary classification or shallow ML and DL approaches [15], [16]. Because each type of threat follows a different attack procedure, multi-classification is getting attention because it aids in selecting prevention techniques [17]. Also, ensemble approaches were utilized to get better accuracy than baseline ML and DL approaches [4]. To address these challenges, in our previous work, we utilized CIC-DDoS2019 and proposed a DL-based stacking ensemble approach to identify and categorize various types of DDoS attacks [18]. Moreover, we used Random Feature Elimination (RFE) with Support Vector Machine (SVM) for feature selection and CNN as the meta-learner in the ensemble model. In this proposed ENIDS (Ensemble Network Intrusion Detection System) model, we focused on 12 different cyberattacks instead of focusing on only DDoS and utilized two separate network traffic datasets, UNSW-15 and CICIDS-2017. Moreover, we used RFE with cross-validation for feature selection. Random Forest (RF) is used as the base algorithm to identify the optimal features, which are model-independent and provide robust and generalized feature importance across different ML/DL models and datasets. RF performs well on large and complex datasets with linear and non-linear data; however, SVM can handle non-linear data by utilizing kernel functions like the radial basis function (RBF), which has a higher computational complexity than RF. Moreover, RF supports multi-class classification by aggregating the votes from multiple decision trees, which makes it robust, whereas SVM is naturally a binary classifier [19]. In our experiments, for large and complex datasets, such as UNSW-15 and CICIDS-2017, containing both linear and nonlinear features, RF's ensemble nature makes it more robust and computationally efficient than SVM for our multi-class classification tasks. We also introduced data resampling techniques to resolve the imbalanced dataset problems by undersampling the majority class and oversampling the minority class, which enhanced the model generalization and prevented biased learning. In our proposed ENIDS model, we used an integrated stacking ensemble model composed of two learners: a base learner

and a meta learner. Stacking ensemble is a learning technique that first trains multiple diverse base learners, combines the prediction output of the base learner, and then uses a meta-learner that makes predictions based on the outputs of the base learner [20]. In ENIDS, merging the pre-trained weights of the base learners and not updating their weights while training meta-learners with a simple DNN architecture reduces the overall computational overload of the proposed ENIDS model compared to previous work. The main contributions of this work are as follows:

- This paper proposes a novel integrated stacking ensemble framework called ENIDS for detecting intrusion in network traffic. ENIDS incorporates three different DL algorithms, namely CNN, LSTM, and GRU, as the base learners to learn the different patterns of the network traffic, these base learners are trained independently, and their training weights are saved for future use.
- ENIDS utilizes the training weights of the three base learners (CNN, LSTM, and GRU) and, after concatenation of the weights, passes them directly as input for the meta-learner, eliminating the need for generating a separate training dataset for the meta-learner. Then the meta-learner is trained again without changing the weights of the base learners. We used DNN as a meta-learner, which is computationally more efficient during training and testing than CNN, and it can effectively integrate and learn from the diverse patterns produced by the base learners.
- The paper addresses the class imbalance problem in multi-class classification by employing data resampling techniques. Specifically, after selecting optimal features using SMOTEENN, ENIDS applies oversampling to the minority class and undersampling to the majority class. This approach helps to mitigate false positive rates commonly associated with class-imbalanced datasets.
- Our proposed ENIDS model is trained using two public network traffic datasets, namely UNSW-15 and CICIDS-2017. These datasets represent real-world network traffic data, providing a realistic environment for assessing the performance of the intrusion detection system.
- The study compares the performance of the ENIDS model with other state-of-the-art methods for network intrusion detection. The evaluation includes metrics such as accuracy, precision, recall, and F1-score, allowing for a comprehensive comparison of the proposed framework with existing approaches. Additionally, the paper presents the execution times of the different methods, providing insights into their computational efficiency.

II. LITERATURE REVIEW

This chapter will review the related literature of previous works in the network intrusion detection system (NIDS) field as well as pinpoint the shortcomings of existing studies, which is the motivation for this research. This section discusses three distinctive phases of developing a NIDS: statistical-based approach, machine learning-based approach, and deep learning-based approach.

A. Statistical-Based Approach

In 1988, Stephen Smaha introduced “Haystack”, an intrusion detection strategy that could notify any unusual events in a multi-user computer system [21]. Haystack was designed to detect any inappropriate user behavior as an infiltration that could compromise an organization’s security and administrative policies. Haystack was inspired by a previous study proposed by Jim Anderson in 1980, in which the system scanned a computer’s daily operations data to uncover usage trends [22]. Both Haystack and Anderson’s work used a statistical approach, relying on massive trail files compiled from all of the business’s computers. Some big IT-based companies began using intrusion detection systems to safeguard their networks around the beginning of 1990. The intrusion detection expert system (IDES) was used as a basis model by AT&T Bell Labs while developing its IDS. Nong Ye proposed a multivariate statistical analysis to detect host-based intrusion in 2002, where he recorded normal activities for a long time and utilized them to detect abnormal activities [23]. To analyze the normal activities and determine the anomalies, this work used the T^2 test proposed by Hotellings and compared the performance with the Chi-squared distance test. In 2007, Sathish Kumar and other researchers provided a methodology for analyzing raw network data in which thresholds are set using the six-sigma technique, and these thresholds are used to identify the network’s normal, uncertain, and anomalous states [24]. The lack of real-time network traffic resources made developing an IDS difficult in the early stages. However, developing an efficient IDS system requires a well-labeled dataset, and the University of California’s publication of the KDD-99 [25] intrusion detection dataset paved the way for researchers to detect network intrusion in a more sophisticated manner using data mining and machine learning (ML) techniques.

B. Machine Learning-Based Approach

Machine learning (ML) is a broader term that involves the development of algorithms or statistical models that use computational methods to enable computer applications to learn from experience and improve their experience in predicting outcomes without being explicitly programmed. These models take historical data as input, train software applications to make decisions, and give prediction results on new data. It is now used in every sector, including healthcare, finance, transportation, marketing, cybersecurity, and natural language processing, and is revolutionizing various other fields [26].

In 1999, Chris Sinclair used an ML-based expert system to automatically classify a network connection by utilizing network patterns [27]. They employed a genetic algorithm and Decision Tree (DT) to develop the IDS to detect “low and slow” attacks, which may contain intrusion behavior. Zhang et al. presented a hierarchy-based network intrusion detection system (HIDE) using a hybrid model comprised of perceptrons and backpropagation to distinguish normal and abnormal traffic flows [28]. The HIDE model is composed of three tiers, and each tier has multiple agents to detect network intrusion. Tier-1 pre-processes the network traffic collected by the probe and sends periodic reports about the traffic to

Tier-2. Tier-2 observes the LAN to check network status, and finally, Tier-3 receives data from both Tier-1 and Tier-2. After pre-processing, all data are sent to the statistical processor to be converted into stimulus vectors and feed them to the perceptron and backpropagation processor to classify network traffic. Patgiri et al. employed an ML-based approach in an old dataset named NSL-KDD to detect cyberattacks generated from Internet of Things (IoT) devices, cloud computing, and social networking sites [29]. Their research focused on comparing the effectiveness of different ML models in detecting four different types of cyberattacks, and they concluded that the classification algorithms they utilized, Random Forest (RF) and Support Vector Machine (SVM), can detect cyberattacks effectively.

Eskin et al. first presented a network intrusion detection technique using unsupervised ML methods in 2002. It includes clustering algorithms, an SVM, and the K-Nearest Neighbor (KNN) algorithm [30]. Their research introduces a geometrical paradigm for unsupervised anomaly detection that maps typical metadata into a feature space. Chih-Fong Tsai and his research team reviewed 55 ML-related intrusion detection systems developed between 2000 and 2007 [31]. They divided the system into single, hybrid, and multi-classifiers and compared them based on the datasets, classification method, and experimental setup. Their findings conclude that KNN and SVM are more popular single classifiers, integrated-hybrid classifiers are the most commonly used hybrid classifiers, and ensemble techniques did not receive much attention. Zhang et al. [32] introduced a hybrid NIDS based on Random Forest (RF) that can detect misuse, patterns of intrusions, and outliers. The authors used an unsupervised learning technique to train the anomaly detection components to detect anomalies and outliers in network traffic flow. Finally, their work combines misuse detection with anomaly detection, allowing anomaly detection to detect novel cyberattacks while misuse detection filters out known intrusions. Sangkatsanee et al. [33] developed a real-time IDS named RT-IDS, which could distinguish between normal traffic and anomalies. The authors identify 12 essential features from network traffic that are important for detecting network anomalies, and several ML algorithms such as DT, and Neural Network (NN) with back-propagation are used, where DT with the Rippler rule outperforms all other algorithms.

Injadat et al. [15] introduced a multi-level optimum ML approach for network anomaly detection. Their strategy showed a 99% detection accuracy and reduced the number of false positives by 1% to 2% using the CICIDS-2017 and UNSW-15 datasets. Disha et al. [16] used Gini Impurity-based Weighted RF to select features from UNSW-15 and Network TON_IoT datasets and compare the performance of different ML models with different feature sets. Magan-Carrion et al. [34] proposed an ML-based intrusion detection system to identify the cyberattacks from the network traffic dataset called UGR’16 generated from heterogeneous devices. Das et al. [35] performed ensemble feature selection and used ensemble ML techniques to detect different types of cyberattacks in the CICIDS-2017, UNSW-NB15, and NSL-KDD datasets. They proposed ensemble feature selection (EnFS), which achieved better accuracy

and F1 scores in all three datasets; however, they performed binary classification, which did not resemble the detection rate of different types of cyberattacks. In their other study, Das et al. [36] developed an ensemble framework for feature selection (FS) methods that aimed to generate an optimal set of features. They also conducted a comparative analysis of various existing approaches and focused on accuracy and false positives using the NSL-KDD dataset.

C. Deep Learning-Based Approach

Deep learning (DL) is a subset of ML that uses Artificial Neural Network (ANN) techniques that are inspired by the composition and operations of the brain. **The advantage of DL is that it includes many layers of neurons that have the ability to identify features, resulting in minimal manual intervention for feature engineering.** As a result, DL will be much more effective than classical ML at identifying complicated and dynamic patterns in network traffic. However, because network traffic usually creates voluminous data, it requires significant processing resources for training and deployment.

A research team from the University of Toronto led by Geoffrey Hinton proposed ImageNet and addressed the fact that DL can outperform any ML algorithms in image classification tasks [37]. In the LSVRC-2010 contest, they trained 1.2 million images using Convolutional neural networks (CNN) with thousands of features. They achieved a lower error rate than any other advanced ML model in the ILSVRC-2012 contest, where they achieved only a 15.3% error rate in the test data, which outperformed every other model. In 2015, LeCun et al. proposed that traditional ML cannot process raw forms of natural data like pixel values of images, matching news items, user interests in a particular product, and many more [38]. However, DL makes up multiple levels of representation layers, which take raw input in one representation layer and transform it into a higher layer that can learn complex functions more abstractly.

In 2016, Javaid et al. proposed a DL-based strategy named STL (self-taught learning) to detect network anomalies using the NSL-KDD dataset [39]. Their STL approach can learn about features from different network sources. These pre-processed features are passed through the auto-encoder and regression with the SoftMax function to classify normal and attack traffic. Their experiments outperform some other research conducted using DL techniques. The authors used the KDD Cup 1999 (KDD-99) dataset for their IDS by using a DL-based method named Deep Neural Network (DNN) and an ML-based method called SVM, where the accuracy of DNN is 15% higher than SVM. Vinayakumar et al. [40] proposed a hybrid DNN called the scale-hybrid-IDS-AlertNet framework that is both scalable and capable of identifying real-time cyberattacks by analyzing network traffic and host-level events. In a related study [41], the authors evaluated the efficacy of a variety of shallow and deep networks for use in NIDS, utilizing the KDD-99 dataset. Finally, Vinayakumar [42] presented a DL-based botnet detection system designed to detect and classify domain names, which can be implemented at the Internet Service Provider (ISP) level to monitor IoT

devices. Before 2016, most of the researchers used KDD-98, KDD-99, and NSL-KDD datasets for their decade-old IDS, which do not match the current network scenario. This dataset crisis was solved when, in 2015, Moustafa et al. published the UNSW-15 network flow datasets, which have both normal and anomalous network traffic [13]. The UNSW-15 network dataset is more useful than previous datasets to evaluate NIDS perfection because it represents contemporary network traffic contexts.

Hanif et al. utilized artificial neural networks (ANN) to detect network intrusion on IoT devices, and in their experiment, they used the UNSW-15 dataset [43]. In their experiment, instead of predicting different types of network attacks, they only detected the normal attributes of the traffic. In order to examine network traffic characteristics of IoT devices in 2018, Moustafa et al. [44] utilized the UNSW-15 dataset, where the authors used AdaBoost ensemble algorithms to discern between normal and abnormal traffic. To construct the NIDS dedicated to identifying attacks in IoT networks, this framework focuses on Domain Name Systems (DNS) and Hypertext Transfer Protocols (HTTP), together with MQ Telemetry Transport (MQTT) and associated flows. They merged three techniques in their framework: ANN, Naive Bayes, and DT, and then passed them through AdaBoost ensemble methods. The DL-based NIDS got more attention from the research community when the Canadian Institute of Cybersecurity (CIC) published two large network traffic datasets, namely CICIDS-2017 [14] and CSE-CIC-IDS-2018 [45].

In 2021, Aleesa et al. [46] utilized UNSW-15 datasets and proposed an AI-based approach to detect network anomalies from the traffic flow. The authors suggested a framework with two levels: level 1 could determine whether a traffic flow was normal or abnormal, and level 2 could classify the many forms of attacks when abnormal traffic was discovered. In their work, instead of incorporating any DL models, shallow ML models are used. In order to evaluate DL and ML-based methods for detecting network anomalies, Liu et al. [47] presented a taxonomy for NIDS. The proposed taxonomy in this work can answer several questions about feature selection, data type selection to predict certain types of attacks, as well as ML and DL model selection based on the type of network data available. Rios et al. [48] made a comparison between incremental and non-incremental broad learning systems (BLS) to identify DoS attacks on the communication networks using the CSE-CIC-IDS-2018 and CICIDS-2017 datasets, where they concluded that in both datasets, the non-incremental approach gives better accuracy; in contrast, the incremental process takes less training time. Faker et al. [49] employed big data as well as DL methods and used UNSW-15 and CICIDS-2017 datasets to evaluate the performance of the models. They incorporated DNN, Random Forest (RF), and Gradient Boosting Tree (GBT), where DNN shows better performance in both datasets for multiclass classification and GBT shows better performance in CICIDS-2017 during binary classification compared to DNN. Henry et al. [50] employed bidirectional DL methods and showed a performance comparison between traditional ML and DL models.

TABLE I: COMPARATIVE OVERVIEW OF THE LITERATURE

Classification Methods	Feature Selection	Pre-processing	Dataset	No. of Classes	Remarks
Statistical Approach [22]	Six sigma techniques	Not mentioned	DARPA	3	Considered only a statistical approach, and not feasible for large datasets.
Random Forest, SVM [29]	RFE	One Hot Encoding	NSL-KDD	4	In this model, RF showed better accuracy when feature selection was avoided. However, SVM performed better when feature selection was applied. The feature selection results in performance degradation for RF and almost the same accuracy for SVM. Considered very old datasets that do not resemble today's attack scenario.
Unsupervised Machine Learning [32]	Random Forest, Information Gain	Not mentioned	KDD-99	3	The performance of the proposed attack detection method tends to decrease as the number of attack connections increases. There is an assumption that the majority of network activities are normal, which may not always be the case. This suggests a potential vulnerability of the system to high-volume attacks.
KNN, RF [15]	Information Gain, Correlation-based	Z-score Normalization, SMOTE	UNSW-NB15, CICIDS-2017	2	The proposed framework, despite reducing time complexity in some areas, still has an overall complexity of $O(MN^2)$ which can be significant for large datasets. The complexity is due to the feature selection stage that requires calculating all the class-feature and feature-feature correlations.
Multiple ML and DL models [16]	Gini Impurity based on Weighted Random Forest	Feature Scaling, label encoding	UNSW-NB15, Network TON_IoT	2	By using Gini impurity as the splitting criterion and adjusting weights for different classes, the technique enables the learning algorithm to better understand the class distribution. This system is tailored for an imbalanced class distribution as it identifies important and relevant features by calculating feature importance scores. However, it only offers binary classification.
Ensemble [35]	Ensemble Feature Selection	Minmax, Data Transformation, Normalization	CICIDS-2017, NSL-KDD, UNSW-NB15	2	The efficiency of ensemble feature selection techniques, such as identifying significant features, eliminating non-discriminating features, and maximizing the synergy created by combining weak features, is the main focus of the proposed system. It employs Logistic Regression, SVM, DT, Naive Bayes, and Neural Networks to classify network traffic into either malicious or benign categories. Though this method includes outdated and recent datasets for training, it shows better accuracy on recent datasets.
Ensemble [36]	Ensemble Feature Selection	Minmax, Normalization, and Transformation	NSL-KDD	2	The system only focuses on dynamic feature selection, which is an ensemble method that incorporates wrapper-based, filter-based, and embedded methods. It combines seven feature selection methods through majority voting and offers data dimension reduction that increases accuracy.
Non-symmetric Deep Auto-Encoder [39]	Information Gain	1-to-n Encoding	NSL-KDD	4	It proposes STL, which is a deep learning approach used for classification consisting of two stages. In the very first stage, it involves in learning feature representation from a large collection of unlabeled data (unsupervised feature learning, or UFL). In the next stage, it applies this learned representation to labeled data for the classification task. While STL can handle data from different distributions, there's an implicit assumption that the unlabeled and labeled data must be relevant to each other. This might limit the methodology's effectiveness when dealing with highly diverse or evolving data distributions. Also, classification accuracy declined sharply when applied to testing data for multi-classification.
DNN [40]	Feature Set Reduction Techniques	Normalization	KDD-99, NSL-KDD, UNSW-NB15, Kyoto, WSN-DS and CICIDS-2017	18	Proposed a real-time hybrid IDS named SHIA that uses distributed DL models and can alert to possible cyberattacks. Perform experiments in various datasets, select features using minimal feature sets, and utilize DNN to detect different types of cyberattacks. However, their system did not mention the computational challenges of the real-time system.
DBN (Deep Belief Networks) [41]	Feature Set Reduction Techniques	Normalization	KDD-99, and NSL-KDD	5	Performed experiments to model the connection records of transmission control protocol/internet protocol (TCP/IP) by utilizing shallow ML and deep belief networks (DBN) and created three different minimal feature sets utilizing feature set reduction techniques. However, no pre-processing techniques are mentioned except normalization, and the network traffic datasets they used are outdated.
Ensemble [44]	Correlation Coefficient	Not mentioned	UNSW-NB15, NIMS botnet	9	The proposed model extracted categorical features from the DNS and HTTP protocols and evaluated its efficacy on the derived feature set from two different datasets. This seems to be an impressive approach; however, it is only limited to DNS and HTTP protocol.
ANN, DNN, RNN, LSTM [46]	Not mentioned	Normalization	UNSW-NB15	9	Experiments are performed in only one dataset and no feature selection approach is obtained.
DNN, RF, GBT [49]	K-means Clustering with homogeneity Metrics	label Encoding, Data Normalization,	UNSW-NB15, CICIDS-2017	2	It obtained both binary and multi-class classifications and presented high accuracy. However, in multi-class classification, it removed benign data, which, if included, could result in declining accuracy.
RNN, GRU [50]	Pearson's Correlation Coefficient	Not mentioned	CICIDS-2017	14	Experiments are performed on only one dataset.

Unlike the above-mentioned research efforts, we used two different recent network traffic datasets, UNSW-15 released by the University of New South Wales [13], and CICIDS-2017 published by the Canadian Institute of Cybersecurity (CIC) [14], to train our proposed DL-based ENIDS and analyze how it performs in different attack scenarios. Many researchers have proposed ML and DL-based techniques to detect and predict network anomalies. Moreover, some researchers proposed DL-based methods, but most of them were focused on binary classification and used shallow ML and DL methods. Moreover, some researchers used an out-of-date dataset where

traffic patterns no longer resemble today's diverse network traffic [51]. In our proposed ENIDS, we used a data resampling technique to balance the imbalanced data samples and employed a stacking ensemble technique composed of different DL models to detect the different kinds of cyberattacks.

Table I presents an analysis of prior research in this field, intending to identify areas where further investigation is needed. The analysis compares six different factors among various studies, namely classification methods, feature selection, data pre-processing, the dataset used, the number of classes, and overall remarks.

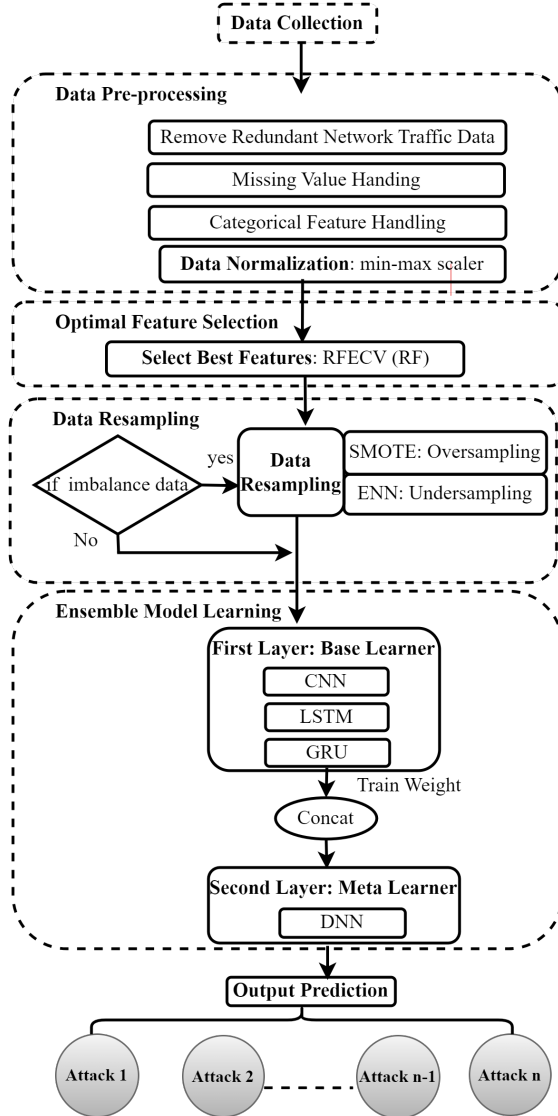


Fig. 1: The framework of the proposed NIDS.

III. METHODOLOGY

A. Proposed ENIDS Overview

This work aims to propose an ensemble NIDS framework to identify several types of cyberattacks that will interrupt network services. The proposed framework comprises four phases: dataset collection, data pre-processing and normalization, optimal feature selection, data resampling, and base and ensemble model learning. Fig. 1 demonstrates the overall overview of the proposed DL-based NIDS framework to detect cyberattacks from network traffic. In the overall experiment, we first performed multi-step data pre-processing. In section III-B, we discuss the utilized datasets in developing the NIDS and perform the first step of data pre-processing to remove the data redundancy as well as improve data quality; the dataset and data pre-processing are discussed in sections III-B.1 and III-B.2, respectively. Moreover, we also perform the data normalization method using a min-max scaler to handle outliers, which are discussed in section III-B.3. The second phase will perform the optimal feature selection to

select the appropriate features for specific types of attacks. Random feature elimination (RFE) is used with 5-fold cross-validation, where it defines two options to remove irrelevant features: select the number of features and utilization of a base algorithm to select the features based on their importance. The feature selection approach is discussed in section III-C. In the third phase, we perform data resampling techniques, to solve the data imbalance classification problems. The data resampling method dynamically oversamples and under-samples the data samples to balance the underrepresented minority class in the datasets. The synthetic minority oversampling technique (SMOTE) [52] is used to oversample the minority class, and then the edited nearest neighbor (ENN) [53] is used to undersample the majority class. The details of the data resampling are discussed in section III-D. The final phase in section III-E consists of base model learning and ensemble model learning. We use three base learners in our experiments, including CNN, LSTM, and GRU, which resided in the ensemble model's first layers. The three base learners are trained with the pre-processed data, and their training weights are saved; their performance with the test data is recorded. The three base learners are discussed in section III-E.1. The training weight of the three base learners is concatenated and passed to the second layer of the ensemble model, which is a DNN, and then the ensemble model is trained again. The details of the ensemble model have been discussed in sections III-E.2. Finally, section III-E.3 shows the runtime complexity of the proposed ENIDS model.

B. Dataset Description and Preprocessing

The proposed ensemble NIDS was built using two up-to-date datasets, UNSW-15 [13], and CICIDS-2017 [14] which contain the different types of most recent cyberattacks. The attack distributions and the number of features are different in both datasets. Data pre-processing is crucial for ML/DL methods, as the performance of the model largely depends on pre-processing. In network traffic datasets, network availability and traffic failure are common issues that can be solved by pre-processing the data from different extents. Data pre-processing removes noise from data, handles missing values, and reduces redundancy from datasets.

1) *Dataset Description:* The UNSW-15 dataset captures raw packets with the IXIA PerfectStorm tool. The Australian Centre for Cyber Security (ACCS) published this network flow data available for public use, including both normal and malicious traffic. For archiving 100 GB of raw traffic, the tcpdump device is employed. ACCS used three networks with 45 different IP addresses to create the dataset, which took 31 hours to collect. The UNSW-15 dataset is divided into four CSV files and comprises 2.5 million records with 49 features. The dataset is further divided into 82,332 testing records and 1,75,341 training records. Backdoor, worms, generic, exploits, fuzzers, reconnaissance, DoS, and shellcode are among the nine categories of network attacks classified as anomalous traffic. In our experiments, we worked with worms, generic, fuzzers, reconnaissance, DoS, and shellcode. The attack distributions in the UNSW-15 dataset are similar to Fig. 2.

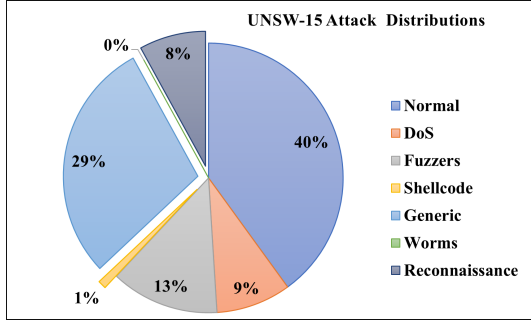


Fig. 2: UNSW-15 dataset attack distributions.

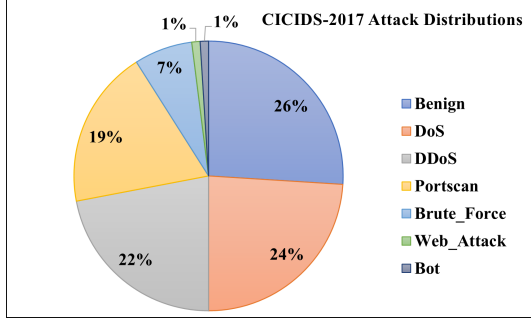


Fig. 3: CICIDS-2017 dataset attack distributions.

The Canadian Institute of Cybersecurity (CIC) published the CICIDS-2017 network traffic data, which contains eight separate CSV files containing five days of regular and aberrant activity of the network flows from Monday to Friday. They build an attack network with a router and a switch for the testbed, and a victim network with a firewall, router, and switches. They produced normal and anomalous traffic using CICFlowMeter software, and 80 features were collected from traffic-generated pcap files. The heartbleed attack, infiltration attack, brute_force attack, DDoS attack, DoS attack, web attack, and botnet are among the cyberattacks covered in CICIDS-2017. It comprises 2,09,417 records, and the attack distributions of the CICIDS-2017 dataset are like in Fig. 3.

2) *Data Pre-processing*: We excluded various features from both datasets that have little impact on normal or abnormal traffic in the initial round of our data pre-processing phase. We remove the following features from the UNSW-15 datasets:

- 1) The 'id' column is removed as it has no impact.
- 2) Time-based features ('stime', 'ltime') are deleted from the UNSW-15 dataset since they are redundant.
- 3) Switch-related information like 'sport', 'srcip', 'dstip', and 'dsport' is also removed from the UNSW-15.
- 4) Categorical features such as 'proto', and 'service' have a wide range of values, thus we employ the label-encoder approach, which generates new, unique numerical values for each category.
- 5) We also eradicate features like 'ct ftp cmd', 'is ftp login', and 'ct flw http method' which have many missing values.

The CICIDS-2017 dataset is pre-processed by considering the following steps:

- 1) Less relevant features such as 'timestamps', and 'IP addresses' were eliminated from the CICIDS-2017.
- 2) As the network flow of this dataset is created using CICFlowMeter, it collects some network-related redundant features such as 'Bwd PSH Flags', 'Bwd Avg Bytes/Bulk', 'Bwd Avg Packets/Bulk', 'Bwd Avg Packets/Bulk', 'Bwd URG Flags', 'Fwd Avg Bytes/Bulk', 'Fwd Avg Packets/Bulk', 'Fwd Avg Bulk Rate', which have many missing values.
- 3) The 'Fwd Header Length' feature is also excluded as it exists twice in the dataset.
- 4) Due to the similarity in network traffic behavior of web attack-brute_force, web attack-xss, and web attack SQL injection, they are labeled as web-attack. Additionally, various forms of DoS attacks, including DoS slowHTTP, DoS hulk, DoS slow loris, and DoS goldeneye, are labeled as DoS.

3) *Data Normalization*: After removing redundant features, we performed data normalization, also known as scaling the features. It is a data pre-processing strategy where all the data transforms into the same scale. In deep learning, normalization is a process of transforming data within a range between 0 and 1. Moreover, when we do not know about the data distribution then normalization is a good approach. If we do not normalize our data that are measured in different scales, they will not contribute equally during model training. Scaling the data equalizes all features, which also assists the algorithm to converge faster, along with optimizing with the gradient descent algorithm. Data distribution is different in both UNSW-15 and CICIDS-2017 datasets, and to fit the data appropriately into the model to get better accuracy we need to normalize our data. As we frequently encounter extremely large or small values among data samples in network traffic data, the min-max scaler is better suited for network anomaly identification due to its high sensitivity to outliers. In our proposed framework, we employed the min-max data normalization technique which scales the data from minimum range 0 to maximum range 1. The mathematical formulation for the min-max scaler is given below:

$$\min - \max = \frac{(x - \min(x))}{(\max(x) - \min(x))} \quad (1)$$

The real-time lowest and maximum values from all the processed data samples are represented here by min and max.

C. Optimal Feature Selection

Feature selection is the process of selecting the most relevant features from a huge number of feature sets and eliminating irrelevant features to improve model performance and reduce the computational cost of the predictive models. Moreover, removing noisy features improves learning efficiency, and reduces the training time of the models. Feature importance determines the contribution of each feature in the prediction model. Selecting important features is critical for developing a better NIDS, as it contains network traffic-related irrelevant information which does not contribute to occurring a cyberattack in the network traffic. In our experiments, we utilized Random Feature Elimination (RFE) with a 5-fold

cross-validation which is easy to configure, and its configuration consists of two options: selecting the number of features and a base algorithm to select the number of features. RFE

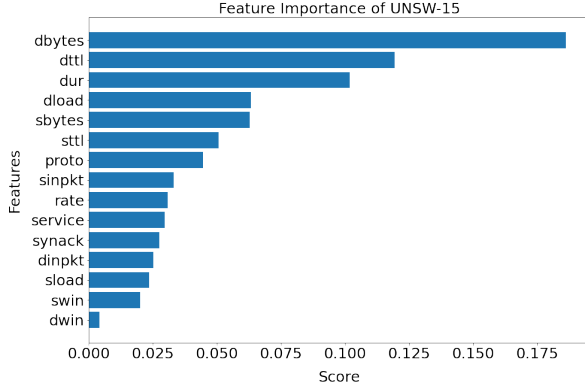


Fig. 4: Feature importance of UNSW-15.

followed both wrapper-style and filter-based feature selection algorithms, where it first searched a subset of features from the entire datasets and removed features until the desired number of features remained. RFE performs k-fold cross-validation, removing characteristics that are less significant for the model with each cross-validation. The process continues until RFE has read all the features from the dataset and only keeps those that improve the overall cross-validation performance. Random Feature Elimination with Cross-Validation (RFECV) is a time-consuming method for selecting features yet yields the best results. In our experiments, we did not define how many features we wanted to select, rather we let the algorithm decide to select the optimum number of features. We used Random Forest (RF), as a base algorithm which is a supervised ML method that employs both the bagging method and DT, to identify the optimal features for our NIDS model. To calculate the soft voting for classification, RF takes the original columns, fits them into decision trees, and then mixes them. The RFECV with Random Forest gave us 25 relevant features from the UNSW-15 dataset and 49 applicable features from the CICIDS-2017 dataset. We also calculated the feature importance using the RF by increasing the purity of the child nodes. One feature gains relevance when the purity of its related child nodes is improved. Each tree estimates the importance of each feature, which is then averaged to obtain the total feature importance. Fig. 4 and Fig. 5 portray the feature importance of the 15 best features from the UNSW-15 and CICIDS-2017 datasets.

D. Data Resampling

In an ML-based approach, imbalanced data is a classification problem. All the classes are not distributed equally in imbalanced data, meaning that the dataset is biased towards one or more classes, with only a few samples for others. As a result, while training a model using imbalanced data, the model was biased toward one or two classes. The majority class is balanced using the undersampling technique, while the minority class is balanced using the oversampling technique. To overcome the imbalanced class problems Synthetic

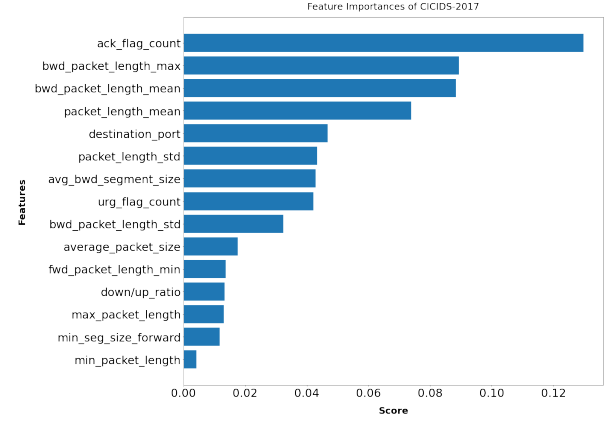


Fig. 5: Feature importance of CICIDS-2017.

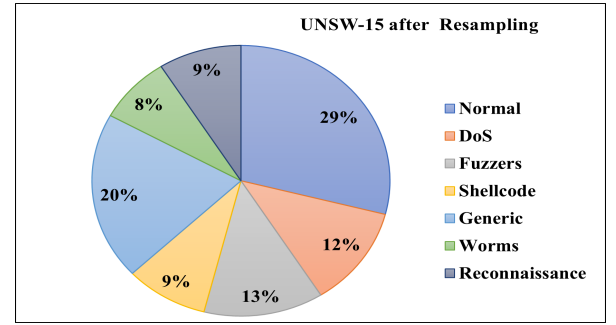


Fig. 6: Attack distribution of UNSW-15 after resampling

Minority Oversampling Technique (SMOTE) is used [52]. The less frequent samples were oversampled using SMOTE, whereas the more frequent samples were undersampled using Edited Nearest Neighbors (ENN). Imbalanced learn, imported as imblearn from the scikit-learn library, is a widely known machine learning library that deals with imbalanced classes. SMOTE takes the following three steps to oversample the minority class:

- 1) It calculates the distance between each sample using Euclidean distance and then modifies these samples using the k-nearest neighbor.
- 2) Then they take n samples and calculate the imbalance ratio from them, as well as the number of samples that need to be made from the samples.

$$n = \text{round}(\text{imbalancedratio}) - 1 \quad (2)$$

$$\text{imbalancedratio} = \frac{S_{\max}}{S_{\min}} \quad (3)$$

Where n is the number of samples.

- 3) Finally, the set of produced samples y is taken from the k-nearest neighbor, and new synthetic samples are constructed from those neighbors.

In order to undersample, the majority class was chosen. The majority of their k-nearest neighbors sample is removed by ENN. If one sample is owned by a most frequent class, and if the classification of the original class is disputed by its three nearest neighbors, it is deleted from the samples; otherwise, it belongs to the minority class. Worms and shellcode traffic

have only 130 (0%) and 1133 (1%) samples in the UNSW-15 dataset, which is relatively low when compared to normal and generic traffic, which include 56,000 and 40,000 samples, respectively. After resampling using SMOTEENN, the total proportion of Worms and Shellcode becomes 9% and 8% respectively. Fig. 6 depicts the attack distribution of the UNSW-15 dataset after data resampling using SMOTEENN. On the other hand, only 1% of samples in the CICIDS-2017 dataset belong to web-attack and bot attack traffic, while benign, DoS, and DDoS attack traffic each include 26%, 24%, and 22% of samples, respectively. After performing the resampling operation, the web-attack and bot became 9% and 10% respectively. Fig. 7 depicts the attack distribution of the CICIDS-2017 dataset after data resampling using SMOTEENN.

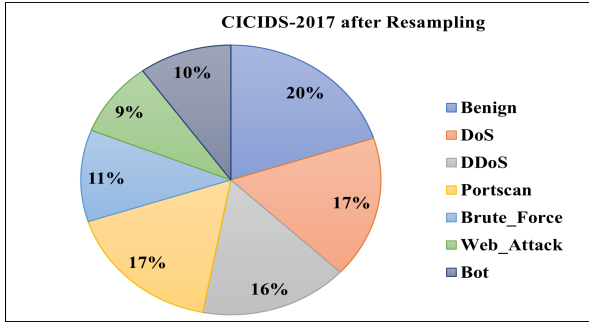


Fig. 7: Attack distribution of CICIDS-2017 after resampling

E. Proposed Classification Model Architecture

Our overall stacking ensemble architecture is comprised of two layers; in the first layer, we employed three different DL-based architectures for our IDS system, which consists of convolutional neural networks (CNN), gated recurrent units (GRU), and long short-term memory (LSTM). We used a deep neural network (DNN) in the second layer, which takes prediction results from the previous three DL models. Each DL architecture has input, hidden, fully connected, and output layers for multi-classification.

1) *Base Model*: A convolutional neural network (CNN), also known as ConvNet, is designed to process grid-like topological data such as images. The first CNN [54] was developed by Yann LeCun, where he proposed LeNet-5, which was able to recognize handwritten characters like postal codes. CNN consists of multiple layers, where earlier layers are responsible for extracting features and later layers combine the features and make predictions. The CNN model consists of convolution and pooling layers that are used for feature extraction, and fully connected layers are responsible for classifying results. Dropout layers are used to prevent the model from becoming overfitted, and the sigmoid or SoftMax activation function is used to derive class-label predictions.

CNN model generally takes an image as input. We reshaped the train and test dataset to (5, 5, 1) dimensions for the UNSW-15 dataset, where the model takes a 5×5 matrix and ImageDataGenerator from the Keras [55] library generates image-type data. For the CICIDS-2017 train, the test is reshaped to (7, 7, 1) dimensions. In our CNN model of the first layer,

we used six two-dimensional filters with a kernel size of (2×2) for the UNSW-15 dataset and a kernel size of (3×3) for the CICIDS-2017 dataset. Every layer of CNN deals with a ReLU activation function. After every two convolution layers, we set one two-dimensional max-pooling layer with different pool sizes. The output is then delivered to fully connected layers, where it is used to train representations of higher-order features that may be used to classify the output into distinct class labels.

Hochreiter and Schmidhuber initially proposed Long Short-Term Memory (LSTM) [56] as an advanced variant of RNN in 1997, to address the problem of exploding and disappearing gradients. The only objective of LSTM is to avoid long-term reliance issues, and it is capable of automatically remembering information for lengthy periods of time. The forget gate of the LSTM cell decides whether the cells need to keep information gained from the previous cell or forget it. The input gate, also known as the store gate, is in control of storing and quantifying new information. A sigmoid function is utilized in the output gate to determine which elements of the cell state will be the output. The LSTM model of the first layer of our proposed stacking ensemble model consists of two LSTM layers: one dense layer, followed by one dropout layer. Like the CNN model, every layer in the LSTM model interacts with a corrected ReLU activation function. The LSTM model's loss function is categorical cross-entropy, and an Adam optimizer with a learning rate of 0.001 was employed for optimization. For LSTM, the input shape is (batch, timestamps, and features), and here batch size means the number of samples we send to the model at a time. For UNSW-15, the timesteps are 5, features are 25 and the input shape is (5, 25); for CICIDS-2017, it is (5, 49).

The Gated Recurrent Unit (GRU) is another short-term memory solution, and its essential principle is virtually identical to that of the LSTM. GRU uses a hidden state to transport information instead of a cell state, and its design consists of two hidden states: the reset gate and the update gate. Tangen's hyperbolic activation function (Tanh), as opposed to CNN and LSTM models, is employed with the GRU model. GRU also takes an input of a 3D tensor with shape (batch size, timesteps, features); here, batch size means the number of samples we send to the model at a time. In UNSW-15, we take 5 timesteps, and 25 features, and the input shape becomes (5, 25); for CICIDS-2017, we take 5 timesteps and 49 features, and the input shape becomes (5, 49). The details of the three base model learning methods using training data and their prediction approach using test data in the proposed framework are shown in Algorithm 1.

We considered the DL-based approach for our proposed ENIDS model to highlight the increasing complexity of network traffic and temporal data patterns. DL techniques are more efficient and robust than ML due to their deep structure, which helps them learn features automatically from raw and large data with multiple classes. The ability to learn dynamic patterns of network traffic with long sequences, the ability to learn both high-level and low-level features, the ability to utilize activation functions to learn non-linearities from complex network traffic data, enhanced detection accuracy, and

reduced false-positive rates. In a comparative study of ML and DL approaches for NIDS, Ahmad et al. [57] mentioned that over the past three years, 60% of the NIDS has been based on DL approaches, 20% of the research is based on a combination of DL and ML models, and only 20% of the research has solely followed ML approaches. Moreover, they visualized among the DL models that RNN, CNN, DNN, and Auto Encoder (AE) were the most used algorithms adopted for developing NIDS. In our experiments, in the base learner, we initially conducted experiments using CNN, LSTM, BiLSTM, CNN-LSTM, and GRU. Then, finally, based on the attack detection rates, training time, false positive rates, and model overfitting rate, we employed CNN, LSTM, and GRU as the base learners.

The primary reasons for selecting CNN, LSTM, and GRU as base learners are given below:

- 1) Although CNN is popular in image processing tasks, it is widely used in network intrusion detection systems [58], [59], and network and IoT data analytics [57], [60]. CNN can capture spatial dependencies in the network traffic patterns, selecting better features by extracting the local correlation of features automatically from the high-dimensional input. The kernel of the convolution layers generates various feature maps, which helps to extract features, and the pooling layer compresses the dimensions of the feature maps without losing important information through local correlations to avoid overfitting [59]. Overall, the ability to recognize network traffic patterns, hierarchically learn features, and be translation-invariant makes CNN effective in detecting normal and malicious traffic behavior.
- 2) Both LSTM and GRU are advanced variants of RNN capable of modeling long-term temporal dependencies, and they work better where the order of data points matters, like sequential data. The architecture of LSTM is more complex than GRU, and LSTM is computationally more expensive than GRU; however, GRU takes less computational resources and has a faster training time [61], [62]. In our study, network intrusions exhibit patterns according to their types, and those patterns can be dispersed across multiple packets over time. Both the UNSW-15 and CICIDS-2017 datasets incorporate a wide range of temporal properties, encompassing shorter sequences and long-term dependencies. In order to identify these patterns effectively, we employed both GRU and LSTM in our proposed ENIDS model. This approach enhances the retrieval of information from temporal features, therefore enhancing the robustness and overall performance of our ensemble model.
- 3) Deep learning algorithms like CNN, LSTM, and GRU have become simpler to train because of recent advancements in optimization mechanisms and Graphical Processing Unit (GPU) support across parallel and distributed computing platforms.

2) *Ensemble Model:* In our work, we are using integrated stacking-based ensemble methods because it uses a second label, also known as a “meta-learner”, through which it can define which classifiers are appropriate and which are not. While

bagging and boosting employ homogeneous weak learners, the ensemble uses heterogeneous weak learners to train them in parallel and aggregate them. The idea of stacking was initially presented by David H. Wolpert in 1992 when he divided the dataset into J equal pieces and utilized J th fold cross-validation during training while the remaining samples were used for testing purposes [63]. He later trained numerous models using the training test pairs as input for the meta-model. Parameter estimates, model selection, and hyperparameter tuning are all part of the stacking framework. Algorithm 1 shows the learning of the meta-learner in the proposed model, where it first concatenates the training weight of the base learners and passes them as input to the meta-learner. Fig. 8 shows the architecture of the proposed stacking ensemble technique.

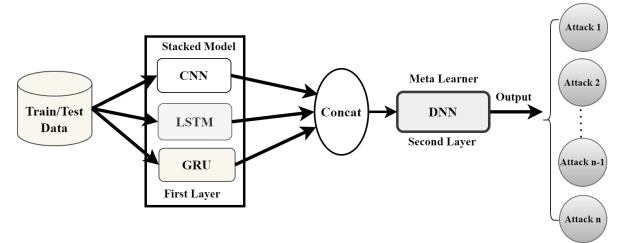


Fig. 8: The stacking ensemble architecture.

In our proposed hybrid deep learning model, called the stacking ensemble model, the meta-learner in the second layer is composed of a Deep Neural Network (DNN). All the results obtained from the first three DL models in the first layer are concatenated and delivered to the DNN model. The non-linear nature of deep learning neural networks can pose difficulties in learning specific features from large datasets, leading to a reduced predictive capacity of a single deep learning model. Stacking ensembles is a technique that addresses this issue by employing a meta-learner, which takes the output of first-layer feature sub-models, combines them, and trains the meta-learner to make improved predictions. This method enhances the prediction ability and robustness of the model by directly passing the output weights of sub-models to the meta-learner. Previous experiments have encountered challenges in detecting multiple types of cyberattacks using large, imbalanced datasets. In this study, we aimed to enhance predictive performance and reduce variance by utilizing the stacking ensemble technique. Specifically, we applied this technique to two large, imbalanced datasets, where the meta-learner of the stacking ensemble model receives and combines predictions from existing models generated by the first-layer base learners. By doing so, we aimed to improve the model’s ability to accurately predict and classify multiple types of cyberattacks.

The proposed stacking ensemble configuration is shown in Fig. 9. The Ensemble model considers the following steps during the model training:

- 1) The weights of the three first-layer base learners—CNN, LSTM, and GRU—are loaded as a list; each loaded model is used as a separate input head to the ensemble model, and because the base learners are marked as

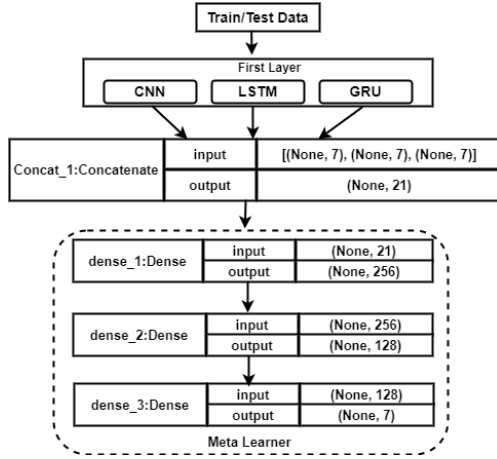


Fig. 9: Configuration of the proposed stacking model.

non-trainable, their weights will not change throughout training.

- 2) The output weight of the three distinct DL models of the base learners merged using a single concatenation merge that creates a single vector of 21 elements from 7 different class labels predicted by each of the 3 models.
- 3) The meta-learner will then analyze the input, employing two hidden layers, and make predictions using an output layer. During the training of the ensemble model, only the weights of the new hidden and output layers will be changed, as the three base learners are marked as non-trainable. Once the model has been fitted, the stacking model is used to predict the unseen data.

Algorithm 1 illustrates the primary learning procedure of the proposed ENIDS model. In the first stage from lines 1–4, we checked the training set IDS_{train} and, if it is imbalanced, performed class resampling, and the resampling strategy is only applied to the training data when it is imbalanced. After that, from lines 5–9, we train the three base learners, BL_1 , BL_2 , and BL_3 , with training data samples in IDS_{train} . Base learner training is the process of training base learners to make predictions on new data using training data. The algorithm concatenates the trained weights gained from each base learner into a list called $Concat-BLW$ after training the base learners. This list is then used to train the meta-learner (ML) in IDS_{train} using the meta-learner training function ($meta-learner_{training}$) on all training data samples. The $Concat-BLW$ list, as well as the training data samples and the weights of the base learners (BL_1 , BL_2 , and BL_3), are used to train the meta-learner. Line 10 concatenates all the training weight gained from the three base learners. The weights of the base learners, on the other hand, are marked as non-trainable, so they are not updated during the training process. Finally, the algorithm makes predictions on the test data samples using the trained base learners and the meta-learner ML . Individual predictions are made using the base learners, and the results are saved in BL_{O1} , BL_{O2} , and BL_{O3} . After that, the meta-learner is used to generate an ensemble prediction, which is saved in ML_P . The algorithm's final output is ML_P , which represents the predicted results for the various types

Algorithm 1 Base and Ensemble Model Learning

Input: IDS_{train} = the training feature set
 IDS_{test} = the test feature set
 $BL = [BL_1, BL_2, BL_3]$; the base learner of the proposed ensemble model, where $BL_1 = CNN$, $BL_2 = LSTM$, $BL_3 = GRU$.
 ML = The meta learner of the proposed ensemble model, which is DNN.
 A_T = The list of different types of cyberattacks
 $BL_w = [BL_{w1}, BL_{w2}, BL_{w3}]$; the trained weight gained from base learners.

Output: ML_P = the prediction results for the different types of cyberattacks in IDS_{test}

- 1: start
- 2: **if** IDS_{train} is imbalance-data **then**
- 3: perform class resampling
- 4: **end if**
- 5: **for all** train data samples $ID_{tr} \in IDS_{train}$ **do**
- 6: $BL_{T1} \leftarrow base-learner_{training}(BL_1, ID_{tr})$
- 7: $BL_{T2} \leftarrow base-learner_{training}(BL_2, ID_{tr})$
- 8: $BL_{T3} \leftarrow base-learner_{training}(BL_3, ID_{tr})$
- 9: **end for**
- 10: $Concat-BLW \leftarrow [BL_{T1}, BL_{T2}, BL_{T3}]$ //Save all the weight gained from three base learners after training.
- 11: **for all** train data samples $ID_{tr} \in IDS_{train}$ **do**
- 12: $ML_T \leftarrow meta-learner_{training}(Concat-BLW, ID_{tr}, BL_{T1,T2,T3} = non-trainable)$ //train meta learner with training data, along with the weight of base learners but the base learner weight will not be updated because it is non-trainable
- 13: **end for**
- 14: **for all** test data samples $ID_{ts} \in IDS_{test}$ **do**
- 15: $BL_{O1} \leftarrow IDS-prediction_{test}(BL_{T1}, ID_{ts})$
- 16: $BL_{O2} \leftarrow IDS-prediction_{test}(BL_{T2}, ID_{ts})$
- 17: $BL_{O3} \leftarrow IDS-prediction_{test}(BL_{T3}, ID_{ts})$
- 18: $ML_P \leftarrow IDS-prediction_{test}(ML_T, ID_{ts})$
- 19: **end for**

of cyberattacks in the test data set IDS_{test} .

3) *Complexity Analysis of Proposed Model:* Runtime complexity is an important metric for assessing algorithm performance and efficiency. Developing an intrusion detection system with a low runtime complexity will aid in performance and reduce the computational resources required. The proposed ENIDS can be trained on a high-speed server; however, the proposed model's deployment may differ depending on the system where it is deployed.

The class resampling operation in line 3 will take $O(N)$, where N is the number of samples in the training set. Lines 5–9 will execute N times, and each base learner training operation in lines 6–8 will take $O(T)$, where T is the time required to train the base learner on one sample. As a result, the time complexity of lines 5-9 is $O(N * T)$. Because it is a simple concatenation operation, the $Concat-BLW$ operation in line 10 will take $O(1)$ time. Lines 11–13 will execute N times, and line 12 will take $O(M)$, where M is the time required to train the meta-learner on one sample. As a result, the time

complexity of lines 11-13 is $O(N * M)$. Lines 14-19's for loop will run M times, and each IDS-prediction test operation in lines 15-18 will take $O(P)$, where P is the time it takes to predict one sample. Thus, the time complexity of lines 14-19 is $O(M * P)$.

Therefore, the total time complexity of the given algorithm will be $O(NT + NM + M * P)$. The actual time taken by the algorithm will be determined by the values of N , T , M , and P .

IV. PERFORMANCE EVALUATION

The proposed ENIDS framework is trained with two publicly available datasets, namely UNSW-15 and CICIDS-2017. Each of the datasets is pre-processed, and an optimal number of features is selected from both datasets. Each dataset contains the seven most recent types of cyberattacks. To train the proposed AI-based ENIDS, we divided our datasets into three parts: the training set, the testing set, and the validation set. We used the stratified training-test split of the Scikit-Learn library. The stratified train-test split divided the data samples into similar proportions. A total of 70% of data samples are used for training the models, while 10% of the data samples are used for validation and the remaining 20% of the data samples are used for testing purposes.

A. Software and Hardware Requirements

In this experiment, a Windows 10 PC with an AMD Ryzen 9 5900HX processor is used. It has 16 GB of RAM, 512 GB of solid-state drive (SSD), and 4GB of NVIDIA GeForce RTX 3050 graphics processing unit (GPU). The NIDS model is implemented using different libraries of Python 3.7, including Keras [55], TensorFlow 2.8.0 [64], and Scikit-learn [65]. The Pandas package for data analysis, NumPy for numerical analysis, and Matplotlib and Seaborn to generate graphs for the experiment results.

B. Evaluation Metric

In order to evaluate the performance of the proposed NIDS, we used a confusion matrix, which is a graphical depiction of the performance of a classification problem and gives the output in matrix format. Using a confusion matrix, a classification model's outcomes can be characterized as true positive (TP), true negative (TN), false positive (FP), and false negative (FN). A classification model's outcomes can be characterized as follows using a confusion matrix:

- 1) True Positive: These values are predicted and labelled as positive. For example, if traffic is forecasted as a DoS assault in our ENIDS system and it is indeed a DoS attack, we can conclude that the ENIDS made an accurate prediction.
- 2) True Negative: These are the values labeled as negative and also predicted as negative and correct. For instance, in the ENIDS model, if traffic is predicted not as a DoS attack and it was not a DoS attack, then the model correctly predicted the traffic.
- 3) False Positive: This occurs when a model predicts a positive value for a class, but the actual value is negative.

For example, if our ENIDS model predicted a DoS attack but it turned out to be typical traffic, we'd have a false-positive result.

- 4) False Negative: This is used to describe results that were anticipated to be negative but ended up being positive. For example, if our ENIDS model projects a packet as regular traffic, however, the traffic was actually a DoS attack. Higher false-negative numbers indicate a defective model.

Fig. 10 shows a perceptible representation of a binary classification confusion matrix. The confusion matrix for multi-class classification will be the same size as the number of classes the model must predict. In our experiments, we used

	Predictive Positive	Predictive Negative
Actual Positive	True Positive	False Positive
Actual Negative	False Negative	True Negative

Fig. 10: Confusion matrix.

two imbalanced datasets, and it is difficult to obtain accurate results from the imbalanced data. The classification accuracy does not always correspond to the model's real performance, especially when the misclassification rate for minor classes is high. It also ignores the problem of class imbalance in a dataset, which occurs when the number of positive and negative levels is vastly different. As a result, various performance evaluation indicators must be considered to achieve actual model performance. In order to get a better overview of the performance of our proposed ENIDS, we considered four important performance metrics for multiclass classification: accuracy, recall, precision, and F1-score. The term accuracy is a ratio of the total number of actual predictions made by the model to the total number of predictions. Precision is determined by dividing the true positives predicted to belong to a specific class by the total number of positive outcomes predicted by the classifier, and recall is a metric that measures how well our model detects true positives. Finally, the F1-score is used to determine how well our model detects actual positives. It's calculated by dividing the number of real positive results by the total number of positive values. The training and testing times of the models are also used to measure the efficacy of the proposed ENIDS.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (4)$$

$$Precision = \frac{TP}{TP + FP} \quad (5)$$

$$Recall = \frac{TP}{TP + FN} \quad (6)$$

$$F1Score = 2 \frac{Precision * Recall}{Precision + Recall} \quad (7)$$

C. Experimental Results and Discussion

Three different DL models, including CNN, LSTM, and GRU, are used in the first layer of the proposed stacking ensemble-based NIDS, and their training weights are used as the input for the meta learner in the second layer, which is a DNN. The ensemble model, as well as the base learners of the first layer, are evaluated with the test set to make a comparison with the proposed ENIDS.

Table II presents a performance comparison of the proposed Ensemble Network Intrusion Detection System (ENIDS) with single deep learning (DL) models and some previous state-of-the-art works on the UNSW-15 dataset proposed in [35], [43], and [46]. ENIDS achieved the best overall performance of the models in the table, with an accuracy of 90.64% and an F1-score of 90.50%. This is superior to the performance of the individual DL models, including the Convolutional Neural Network (CNN), Long Short-Term Memory (LSTM), and Gated Recurrent Unit (GRU), which attained accuracy scores ranging from 88.4% to 89.1% and F1-scores ranging from 88.2% to 89.1%. ENIDS outperformed the other models due to its ability to integrate multiple DL models into an ensemble framework, allowing it to capture different aspects of network traffic and make more informed decisions by combining the predictions of the base models. Additionally, the ENIDS model used a meta-learner in the second layer to learn how to combine the weights of the base models to make a final prediction, which improved the model's accuracy. ENIDS requires more training and testing time (460 and 10.4 seconds, respectively) compared to the base learners due to the additional steps involved in combining the multiple models and processing their predictions.

TABLE II: MODEL PERFORMANCE COMPARISON FOR THE UNSW-15 DATASET

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)	Train Time (s)	Test Time (s)
CNN	89.1	89.7	89.2	89.1	322	7.3
LSTM	89.0	89.5	89.0	89.0	381	7.8
GRU	88.4	89.4	88.4	88.2	364	7.5
EnFS_EnNN [35]	79.7	96.7	77.3	85.9	-	-
ANN [43]	84	-	-	-	-	-
RNN-LSTM [46]	85.38	-	-	-	-	-
Proposed ENIDS	90.6	90.9	90.6	90.5	460	10.4

Furthermore, ENIDS addressed the issue of imbalanced classification by using data augmentation techniques or class weighting, which helped to balance the class distribution and led to improved classification performance. In contrast, the proposed EnFS model in [35] achieved a high precision score of 96.7%, which was higher than the proposed ENIDS model's precision score of 90.94%. However, EnFS is limited to detecting only one type of attack and does not address the issue of imbalance classification. Overall, the superior performance of the proposed ENIDS model is due to its ability to integrate multiple DL models into an ensemble framework, to use a meta-learner to combine the predictions of the base

models, and to address the issue of imbalance classification. These techniques allowed ENIDS to achieve high accuracy and F1-score while still maintaining a relatively high precision score, outperforming the individual DL models and the state-of-the-art EnFS model. Fig. 11 presents a confusion matrix that provides insights into the detection rates of different types of attacks in the UNSW-15 dataset using the proposed ensemble approach. The results show that the ensemble approach achieves high detection rates for several types of attacks, including worms, reconnaissance, and generic attacks, with detection rates exceeding 95%. The proposed model also achieves a detection rate of more than 90% for shellcode and normal traffic. However, approximately 6% of these are misclassified as reconnaissance and fuzzers attacks. The model can detect 89% of DoS attacks but only achieves a 53% detection rate for fuzzer attacks, with 32% of them misclassified as normal traffic. This is because fuzzers attacks are designed to closely resemble normal traffic, making them difficult to distinguish from legitimate traffic. Moreover, hackers may use fuzzers to simulate normal traffic, further complicating the detection process. Overall, the proposed ensemble approach performs well in detecting most attacks in the UNSW-15 dataset but struggles with identifying fuzzers attacks. Further research is needed to improve the model's ability to detect these attacks, which could include incorporating additional features or developing more advanced detection techniques.

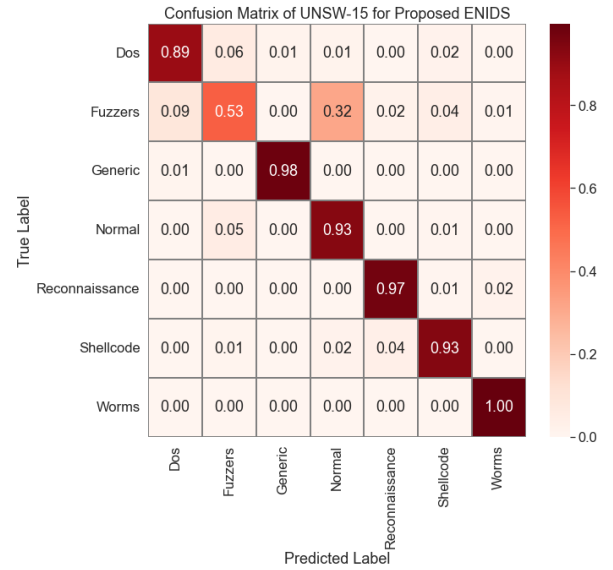


Fig. 11: Confusion matrix of UNSW-15.

Table III presents the performance comparison of different models on the CICIDS-2017 dataset in terms of accuracy, precision, recall, and F1-score. The proposed ENIDS model achieves outstanding results, outperforming all other models in the table. Specifically, the proposed ENIDS model achieves an accuracy of 99.6%, precision of 99.5%, recall of 99.7%, and F1-score of 99.6%. In comparison, the EnFS model [35] achieves an accuracy of 98.9%, which is comparable to the state-of-the-art models in previous works. However, the EnFS model [35] does not address the issue of attack

variations or imbalanced datasets, which can be critical for practical deployment. Moreover, the proposed ENIDS model outperforms other traditional models, such as RBF-BLS [48] and Bidirectional RNN-GRU [50], achieving an accuracy of 96.63% and 98.99%, respectively. The proposed ENIDS model demonstrates a high recall rate of 99.7%, indicating a low false-negative rate and high detection rate. The training and testing times of the ENIDS model for the CICIDS-2017 dataset are 1153 seconds and 15.7 seconds, respectively, which is higher compared to individual base learners due to additional steps required in the ensemble model. The confusion matrix of CICIDS-2017 illustrates that the proposed ensemble NIDS can successfully detect all types of attacks, including Bot, Brute-force, DoS, DDoS, Port scan, and Web-attack, while accurately detecting 98% of benign network traffic. Moreover, Fig. 12 depicts the confusion matrix for the CICIDS-2107 dataset.

TABLE III: MODEL PERFORMANCE COMPARISON FOR THE CICIDS-2017 DATASET

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)	Train Time (s)	Test Time (s)
CNN	97.5	97.7	97.6	97.3	499	8.6
LSTM	97.2	97.4	97.3	97.1	677	12.7
GRU	96.0	96.4	96.1	96.0	637	12.3
EnFS_En NN [35]	98.9	99.9	98.8	99.3	-	-
RBF-BLS [48]	96.63	-	-	96.87	15.60	-
Bi RNN-GRU [50]	98.99	-	-	-	-	-
Proposed ENIDS	99.6	99.5	99.7	99.6	1153	15.7

The proposed ensemble approach requires additional training time compared to the base models, as it involves training both the base models in the first layer and the meta-learner in the second layer. This is because, during training, the ensemble model takes both the training data and the training weights of the base learners from the first layer. The complexity pertains to the importance of maintaining and updating weights for the meta-learner, which is responsible for learning the optimal way to integrate predictions throughout training. However, this extra training time is offset by the improved performance of the ensemble model, as it combines the predictions of multiple models to improve generalization performance. During testing, the proposed ensemble approach requires more time than the base learners, as the test data must pass through the entire ensemble model. The pre-trained ensemble model processes the test data through the base learners before passing it on to the meta-learner, who makes final predictions. These processes increase testing time by requiring more computations to aggregate the predictions of diverse models for improved ENIDS model accuracy and performance. However, the ensemble model can handle different types of network traffic more effectively than the individual base models, as it is able to capture different aspects of the data and combine them to make more informed decisions. Overall, although the proposed ensemble approach requires more training and testing time than the base models, it ultimately leads to improved performance, making it a promising approach for network intrusion detection.

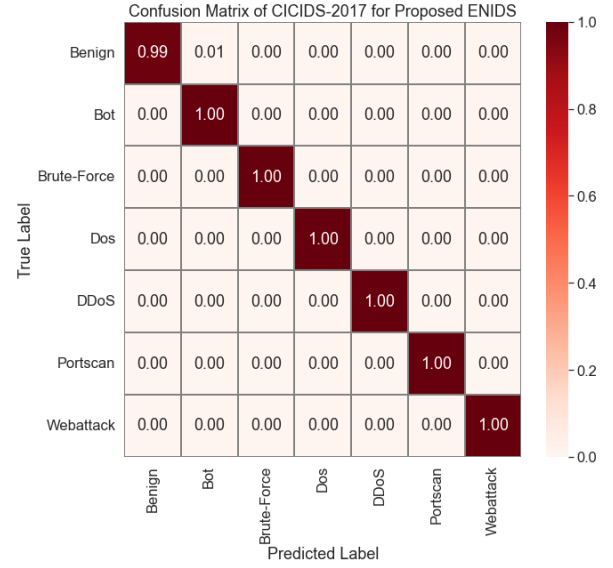


Fig. 12: Confusion matrix of CICIDS-2017.

In our experiments, we used both LSTM and GRU to detect a wide range of temporal features, and we found that LSTM outperformed GRU in terms of detection accuracy on both datasets. This is because LSTM is better equipped to handle larger datasets, and the datasets used in this study were substantially large. LSTM's complex architecture with three gates, including an input gate, an output gate, a forget gate, and an additional memory cell, allows it to capture more long-term dependencies in the input data. On the other hand, GRU has a simpler architecture with only two gates, an update gate and a reset gate, which makes it less effective in capturing long-term dependencies. However, despite LSTM's superior detection accuracy, GRU has a faster training time and is more computationally efficient than LSTM. This is because the GRU's simpler architecture requires fewer calculations and parameters, resulting in faster training times. Therefore, if training time and computational efficiency are more significant factors than detection accuracy, GRU may be a better choice than LSTM.

In summary, ENIDS integrates multiple deep-learning models into an ensemble framework to capture different aspects of network traffic and make informed decisions. A meta-learner combines the predictions of the base models to improve accuracy. ENIDS addresses imbalanced classification with data augmentation techniques or class weighting, outperforming individual models and state-of-the-art models on UNSW-15 and CICIDS-2017 datasets. However, it struggles with identifying fuzzers attacks and requires further research. ENIDS achieves an attack detection rate with unseen data in only 10.4 seconds and 15.7 seconds, respectively for UNSW-15 and CICIDS-2017 datasets, despite longer training time.

V. CONCLUSION

Driven by the advancement in ultra-high speed network technologies such as 5G, the number of connected devices to the Internet is growing rapidly. Hackers/intruders continue to

use new techniques to launch large-scale cyberattacks, making network traffic more vulnerable. This research suggested an AI-based ensemble approach for identifying various types of cyberattacks. In contrast to earlier research, our proposed novel ensemble NIDS model not only determines if the network traffic is benign or normal but also the type of assault in the flow. Compared to the existing models, our suggested stacking ensemble model provides a more accurate forecast. Overall, stacking ensemble DNN, a hybrid deep learning approach, and a well-defined data pre-processing technique are presented in this study. Two well-known datasets, UNSW-15 and CICIDS-2017, were selected for this study because they closely mimic real network traffic flow, which improves the efficiency, robustness, and practicality of our approach. The best possible traffic flow features are selected using random feature elimination with cross-validation (RFECV), and the SMOTEENN technique is employed to resample the imbalanced classes. With the help of the suggested technique, we trained various robust and large deep learning models in the first layer of the proposed model, concatenated their training weights, and then passed them to the second layer of the stacking ensemble technique, which consists of a DNN model. By retraining the model, we can increase the classification accuracy for identifying various cyberattacks. Through the experiments, the proposed ENIDS model achieved higher accuracy of 90.6% and 99.6% in the UNSW-15 and CICIDS-2017 datasets, respectively. Moreover, the experiments show that the proposed model achieves a higher F1-score of 90.5% and 99.6% in both datasets, which outperformed other state-of-the-art approaches.

The main limitation of the proposed ENIDS is that it can't detect new attacks before they are detected since it is constrained by the labels used in the training phase. Additionally, the proposed ENIDS cannot distinguish it from legitimate traffic or other intrusions if new traffic comes in. To overcome this, we will employ the transfer learning technique to assess how well they function on unknown network traffic. Moreover, we will deploy the proposed ENIDS into live network traffic so that the model can analyze the traffic flow of networks and distinguish between normal and malicious traffic.

REFERENCES

- [1] S. Atske, "The internet and the pandemic," Sep 2021. [Online]. Available: <https://www.pewresearch.org/internet/2021/09/01/the-internet-and-the-pandemic/>
- [2] S. Kemp, Jan 2024. [Online]. Available: <https://datareportal.com/reports/digital-2024-global-overview-report>
- [3] Nov 2023. [Online]. Available: <https://www.ericsson.com/en/reports-and-papers/mobility-report/reports/november-2023>
- [4] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, "Survey of intrusion detection systems: techniques, datasets and challenges," *Cybersecurity*, vol. 2, no. 1, pp. 1–22, 2019.
- [5] N. Hu, Z. Tian, H. Lu, X. Du, and M. Guizani, "A multiple-kernel clustering based intrusion detection scheme for 5g and iot networks," *International Journal of Machine Learning and Cybernetics*, pp. 1–16, 2021.
- [6] L. Martin, "Cyber kill chain@," Jun 2022. [Online]. Available: <https://www.lockheedmartin.com/en-us/capabilities/cyber/cyber-kill-chain.html>
- [7] g. checkpoint, "Check point research: Cyber attacks increased 50% year over year - check point software," Jan 2022. [Online]. Available: <https://blog.checkpoint.com/2022/01/10/check-point-research-cyber-attacks-increased-50-year-over-year/>
- [8] S. Imperva, "What is a cyber attack — types, examples prevention — imperva," 2021. [Online]. Available: <https://www.imperva.com/learn/application-security/cyber-attack/>
- [9] i. CSIS, "Strategic technologies program — center for strategic and international studies," Nov 2022. [Online]. Available: <https://www.csis.org/programs/strategic-technologies-program>
- [10] D. Imperva, "Ddos threat landscape report q1 2022 — resource library," 2022. [Online]. Available: <https://www.imperva.com/resources/resource-library/reports/ddos-threat-landscape-report/>
- [11] D. Kwon, H. Kim, J. Kim, S. C. Suh, I. Kim, and K. J. Kim, "A survey of deep learning-based network anomaly detection," *Cluster Computing*, vol. 22, no. 1, pp. 949–961, 2019.
- [12] J. Ahmad, H. Farman, and Z. Jan, "Deep learning methods and applications," in *Deep learning: convergence to big data analytics*. Springer, 2019, pp. 31–42.
- [13] N. Moustafa and J. Slay, "Unsw-nb15: a comprehensive data set for network intrusion detection systems (unsw-nb15 network data set)," in *2015 military communications and information systems conference (MilCIS)*. IEEE, 2015, pp. 1–6.
- [14] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," *ICISSp*, vol. 1, pp. 108–116, 2018.
- [15] M. Injadat, A. Moubayed, A. B. Nassif, and A. Shami, "Multi-stage optimized machine learning framework for network intrusion detection," *IEEE Transactions on Network and Service Management*, vol. 18, no. 2, pp. 1803–1816, 2020.
- [16] R. A. Disha and S. Waheed, "Performance analysis of machine learning models for intrusion detection system using gini impurity-based weighted random forest (giwrf) feature selection technique," *Cybersecurity*, vol. 5, no. 1, p. 1, 2022.
- [17] R. Hu, Z. Wu, Y. Xu, and T. Lai, "Multi-attack and multi-classification intrusion detection for vehicle-mounted networks based on mosaic-coded convolutional neural network," *Scientific Reports*, vol. 12, no. 1, pp. 1–16, 2022.
- [18] M. I. Sayed, I. M. Sayem, S. Saha, and A. Haque, "A multi-classifier for ddos attacks using stacking ensemble deep neural network," in *2022 International Wireless Communications and Mobile Computing (IWCMC)*. IEEE, 2022, pp. 1125–1130.
- [19] M. Fernández-Delgado, E. Cernadas, S. Barro, and D. Amorim, "Do we need hundreds of classifiers to solve real world classification problems?" *The journal of machine learning research*, vol. 15, no. 1, pp. 3133–3181, 2014.
- [20] X. Dong, Z. Yu, W. Cao, Y. Shi, and Q. Ma, "A survey on ensemble learning," *Frontiers of Computer Science*, vol. 14, pp. 241–258, 2020.
- [21] S. E. Smaha et al., "Haystack: An intrusion detection system," in *Fourth Aerospace Computer Security Applications Conference*, vol. 44. Orlando, FL, USA, 1988.
- [22] J. P. Anderson, "Computer security threat monitoring and surveillance," *Technical Report, James P. Anderson Company*, 1980.
- [23] N. Ye, S. M. Emran, Q. Chen, and S. Vilbert, "Multivariate statistical analysis of audit trails for host-based intrusion detection," *IEEE Transactions on computers*, vol. 51, no. 7, pp. 810–820, 2002.
- [24] S. A. P. Kumar, A. Kumar, and S. Srinivasan, "Statistical based intrusion detection framework using six sigma technique," *IJCSNS*, vol. 7, no. 10, p. 333, 2007.
- [25] S. D. Bay, D. Kibler, M. J. Pazzani, and P. Smyth, "The uci kdd archive of large data sets for data mining research and experimentation," *ACM SIGKDD explorations newsletter*, vol. 2, no. 2, pp. 81–85, 2000.
- [26] A. Paleyes, R.-G. Urma, and N. D. Lawrence, "Challenges in deploying machine learning: a survey of case studies," *ACM Computing Surveys*, vol. 55, no. 6, pp. 1–29, 2022.
- [27] C. Sinclair, L. Pierce, and S. Matzner, "An application of machine learning to network intrusion detection," in *Proceedings 15th annual computer security applications conference (ACSAC'99)*. IEEE, 1999, pp. 371–377.
- [28] Z. Zhang, J. Li, C. Manikopoulos, J. Jorgenson, and J. Ucles, "Hide: a hierarchical network intrusion detection system using statistical preprocessing and neural network classification," in *Proc. IEEE Workshop on Information Assurance and Security*, vol. 85, 2001, p. 90.
- [29] R. Patgiri, U. Varshney, T. Akutota, and R. Kunde, "An investigation on intrusion detection system using machine learning," in *2018 IEEE Symposium Series on Computational Intelligence (SSCI)*, 2018, pp. 1684–1691.
- [30] E. Eskin, A. Arnold, M. Prerau, L. Portnoy, and S. Stolfo, "A geometric framework for unsupervised anomaly detection," in *Applications of data mining in computer security*. Springer, 2002, pp. 77–101.

- [31] C.-F. Tsai, Y.-F. Hsu, C.-Y. Lin, and W.-Y. Lin, "Intrusion detection by machine learning: A review," *expert systems with applications*, vol. 36, no. 10, pp. 11 994–12 000, 2009.
- [32] J. Zhang, M. Zulkernine, and A. Haque, "Random-forests-based network intrusion detection systems," *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, vol. 38, no. 5, pp. 649–659, 2008.
- [33] P. Sangkatsanee, N. Wattanapongsakorn, and C. Charnsripinyo, "Practical real-time intrusion detection using machine learning approaches," *Computer Communications*, vol. 34, no. 18, pp. 2227–2235, 2011.
- [34] R. Magán-Carrión, D. Urda, I. Díaz-Cano, and B. Dorronsoro, "Towards a reliable comparison and evaluation of network intrusion detection systems based on machine learning approaches," *Applied Sciences*, vol. 10, no. 5, p. 1775, 2020.
- [35] S. Das, S. Saha, A. T. Priyoti, E. K. Roy, F. T. Sheldon, A. Haque, and S. Shiva, "Network intrusion detection and comparative analysis using ensemble machine learning and feature selection," *IEEE Transactions on Network and Service Management*, 2021.
- [36] S. Das, D. Venugopal, S. Shiva, and F. T. Sheldon, "Empirical evaluation of the ensemble framework for feature selection in ddos attack," in *2020 7th IEEE International Conference on Cyber Security and Cloud Computing (CSCloud)/2020 6th IEEE International Conference on Edge Computing and Scalable Cloud (EdgeCom)*. IEEE, 2020, pp. 56–61.
- [37] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Communications of the ACM*, vol. 60, no. 6, pp. 84–90, 2017.
- [38] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [39] A. Javaid, Q. Niyaz, W. Sun, and M. Alam, "A deep learning approach for network intrusion detection system," in *Proceedings of the 9th EAI International Conference on Bio-inspired Information and Communications Technologies (formerly BIONETICS)*, 2016, pp. 21–26.
- [40] R. Vinayakumar, M. Alazab, K. P. Soman, P. Poornachandran, A. Al-Nemrat, and S. Venkatraman, "Deep learning approach for intelligent intrusion detection system," *IEEE Access*, vol. 7, pp. 41 525–41 550, 2019.
- [41] R. Vinayakumar, K. P. Soman, and P. Poornachandran, "Evaluating effectiveness of shallow and deep networks to intrusion detection system," in *2017 International Conference on Advances in Computing, Communications and Informatics (ICACCI)*, 2017, pp. 1282–1289.
- [42] R. Vinayakumar, M. Alazab, S. Srinivasan, Q.-V. Pham, S. K. Padanayil, and K. Simran, "A visualized botnet detection system based deep learning for the internet of things networks of smart cities," *IEEE Transactions on Industry Applications*, vol. 56, no. 4, pp. 4436–4456, 2020.
- [43] S. Hanif, T. Ilyas, and M. Zeeshan, "Intrusion detection in iot using artificial neural networks on unsw-15 dataset," in *2019 IEEE 16th international conference on smart cities: improving quality of life using ICT & IoT and AI (HONET-ICT)*. IEEE, 2019, pp. 152–156.
- [44] N. Moustafa, B. Turnbull, and K.-K. R. Choo, "An ensemble intrusion detection technique based on proposed statistical flow features for protecting network traffic of internet of things," *IEEE Internet of Things Journal*, vol. 6, no. 3, pp. 4815–4830, 2018.
- [45] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *International Conference on Information Systems Security and Privacy*, 2018.
- [46] A. Aleesa, M. Younis, A. A. Mohammed, and N. Sahar, "Deep-intrusion detection system with enhanced unsw-nb15 dataset based on deep learning techniques," *Journal of Engineering Science and Technology*, vol. 16, no. 1, pp. 711–727, 2021.
- [47] H. Liu and B. Lang, "Machine learning and deep learning methods for intrusion detection systems: A survey," *applied sciences*, vol. 9, no. 20, p. 4396, 2019.
- [48] A. L. G. Rios, Z. Li, K. Bekshentyayeva, and L. Trajković, "Detection of denial of service attacks in communication networks," in *2020 IEEE international symposium on circuits and systems (ISCAS)*. IEEE, 2020, pp. 1–5.
- [49] O. Faker and E. Dogdu, "Intrusion detection using big data and deep learning techniques," in *Proceedings of the 2019 ACM Southeast Conference*, 2019, pp. 86–93.
- [50] A. Henry and S. Gautam, "Intelligent intrusion detection system using deep learning technique," in *Computing, Communication and Learning: First International Conference, CoCoLe 2022, Warangal, India, October 27–29, 2022, Proceedings*. Springer, 2023, pp. 220–230.
- [51] B. Dong and X. Wang, "Comparison deep learning method to traditional methods using for network intrusion detection," in *2016 8th IEEE international conference on communication software and networks (ICCSN)*. IEEE, 2016, pp. 581–585.
- [52] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "Smote: synthetic minority over-sampling technique," *Journal of artificial intelligence research*, vol. 16, pp. 321–357, 2002.
- [53] Z. Xu, D. Shen, T. Nie, and Y. Kou, "A hybrid sampling algorithm combining m-smote and enn based on random forest for medical imbalanced data," *Journal of Biomedical Informatics*, vol. 107, p. 103465, 2020.
- [54] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [55] A. Gulli and S. Pal, *Deep learning with Keras*. Packt Publishing Ltd, 2017.
- [56] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [57] Z. Ahmad, A. Shahid Khan, C. Wai Shiang, J. Abdullah, and F. Ahmad, "Network intrusion detection system: A systematic study of machine learning and deep learning approaches," *Transactions on Emerging Telecommunications Technologies*, vol. 32, no. 1, p. e4150, 2021.
- [58] R. Vinayakumar, K. Soman, and P. Poornachandran, "Applying convolutional neural network for network intrusion detection," in *2017 International Conference on Advances in Computing, Communications and Informatics (ICACCI)*. IEEE, 2017, pp. 1222–1228.
- [59] B. Riyaz and S. Ganapathy, "A deep learning approach for effective intrusion detection in wireless networks using cnn," *Soft Computing*, vol. 24, pp. 17 265–17 278, 2020.
- [60] F. Ullah, H. Naeem, S. Jabbar, S. Khalid, M. A. Latif, F. Al-Turjman, and L. Mostarda, "Cyber security threats detection in internet of things using deep learning approach," *IEEE access*, vol. 7, pp. 124 379–124 389, 2019.
- [61] S. Yang, X. Yu, and Y. Zhou, "Lstm and gru neural network performance comparison study: Taking yelp review dataset as an example," in *2020 International workshop on electronic communication and artificial intelligence (IWECAI)*. IEEE, 2020, pp. 98–101.
- [62] H. Gwon, C. Lee, R. Keum, and H. Choi, "Network intrusion detection based on lstm and feature embedding," *arXiv preprint arXiv:1911.11552*, 2019.
- [63] D. H. Wolpert, "Stacked generalization," *Neural networks*, vol. 5, no. 2, pp. 241–259, 1992.
- [64] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard *et al.*, "{TensorFlow}: a system for {Large-Scale} machine learning," in *12th USENIX symposium on operating systems design and implementation (OSDI 16)*, 2016, pp. 265–283.
- [65] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg *et al.*, "Scikit-learn: Machine learning in python," *the Journal of machine Learning research*, vol. 12, pp. 2825–2830, 2011.



Ibrahim Mohammed Sayem received his Bachelor of Science in Computer Science & Engineering from the University of Chittagong in 2018 and his Master of Science in Computer Science from Western University, London, Ontario, Canada, in 2022. Currently, he is working as a Teaching faculty member at Humber College in Toronto, Ontario. His research interests lie in the areas of network security, machine learning, deep learning, 5G network QoS/QoE, and time series analytics.



Moinul Islam Sayed Moinul is a doctoral candidate at Western University, Canada, specializing in cybersecurity and privacy. His expertise spans multiple domains, including network security, mobile device security, and the security of Internet of Things (IoT) devices. Through his research, Moinul aims to contribute to the advancement of secure computing practices and provide valuable insights for ensuring the resilience and protection of diverse digital ecosystems.



Sajal Saha, Ph.D. is an Assistant Professor in the Department of Computer Science at the University of Northern British Columbia, Canada. His research interests are primarily focused on Internet Traffic Analysis, Cyber-attack Detection, and Data Mining, with special emphasis on traffic prediction, anomaly detection, and association rule mining. He completed his Ph.D. at Western University, Canada, where his research contributed significantly to efficient traffic forecasting and cyber-attack detection. Alongside his academic contributions, he has also gained considerable industry experience as a Research Fellow at Juniper Network and Software Engineer at Samsung Electronics.

erable industry experience as a Research Fellow at Juniper Network and Software Engineer at Samsung Electronics.



Dr. Anwar Haque is an Associate Professor and the Undergraduate Chair of the Dept. of Computer Science at the Western University. Before joining Western, he was an Associate Director at Bell Canada. Dr. Haque's primary research areas of interest include next-gen communication networks (such as 5G & beyond), IoT, and cyber-security, focusing on autonomous systems, smart services and applications. He has authored/co-authored more than 100 peer-reviewed research publications in leading journals and conferences, written many industry

technical papers, holds several patents/licenses, and supervised more than 100 graduate students/highly qualified professionals. He has been awarded several national/provincial-level research grants, including NSERC, MITACS, OCE, and SOSCIP. Dr. Haque's collaborative research grants are valued at more than \$15 million. At Western, he served as the industry expert-in-residence in the Faculty of Science, a member of Western's Senate, and on the inaugural advisory committee for the newly established Bell-Western 5G Research Centre. Dr. Haque is the director of the Western Information & Networking Group (WING) Lab, where he conducts cutting-edge research in emerging network technologies and smart systems.